CRYPTOGRAPHY LAB TUTORIAL

AES Modes of Operation

ECB • CBC • CTR

An in-depth, hands-on comparison with runnable Python code, padding concepts, attack demonstrations and diagrams

What you will build	Files
Padding (PKCS#7) from scratch	`padding_demo.py`
AES-ECB encrypt/decrypt + pattern & reordering attacks	`ecb_demo.py`, `ecb_image_demo.py`
AES-CBC, hand-built chain, bit-flipping, IV experiments	`cbc_demo.py`
AES-CTR, hand-built keystream, nonce-reuse and malleability	`ctr_demo.py`
Side-by-side comparison + production IV/nonce pattern	`compare_modes.py`, `random_iv_demo.py`

Tested with Python 3.12 and the `cryptography` library 46.x. Every output block in this tutorial was produced by actually running the code shown.

1. Introduction and Setup

1.1 Learning objectives

After completing this tutorial you will be able to:

- Explain why a block cipher such as AES needs a **mode of operation** to encrypt real-world messages.
- Apply and remove **PKCS#7 padding** by hand and explain why zero padding is unsafe.
- Describe the data flow of **ECB, CBC and CTR** using diagrams and formulas.
- Run Python programs for each mode and interpret their output.
- Demonstrate concrete weaknesses: ECB pattern leakage, CBC bit-flipping, CTR nonce reuse.
- Choose a suitable mode, and explain why authenticated encryption (AES-GCM) is preferred in practice.

1.2 Environment setup

All programs use the widely used `cryptography` package. Install it once and run each script with `python <script>.py`.

```
$ pip install cryptography numpy matplotlib
$ python padding_demo.py
```

> **Security warning**
> The keys, IVs and nonces in this tutorial are **fixed constants** so that your output matches the printed output exactly. Never hard-code keys, IVs or nonces in real software. Section 8 shows the correct random-generation pattern.

2. AES and the Need for Modes of Operation

2.1 AES in one page

The Advanced Encryption Standard (AES, standardised as FIPS 197) is a **block cipher**: a keyed permutation that maps exactly one 128-bit (16-byte) input block to one 128-bit output block, and can be reversed with the same key. The block size is always 128 bits; only the key length changes.

Variant	Key size	Block size	Rounds
AES-128	128 bits (16 bytes)	128 bits (16 bytes)	10
AES-192	192 bits (24 bytes)	128 bits (16 bytes)	12
AES-256	256 bits (32 bytes)	128 bits (16 bytes)	14

Inside each round AES applies SubBytes, ShiftRows, MixColumns and AddRoundKey (the final round omits MixColumns). For this tutorial we treat AES as a black box: **E_K(block)** encrypts one 16-byte block and **D_K(block)** decrypts it.

2.2 Why a block cipher alone is not enough

Real messages are rarely exactly 16 bytes. A **mode of operation** is the rule that tells us how to apply the one-block primitive to a long message. Every mode must answer three questions:

1. **Chaining:** how are multiple blocks combined? Independently, or linked to each other?
2. **Randomisation:** how do we make sure that encrypting the same data twice does not look the same?
3. **Length:** what happens when the message is not a multiple of 16 bytes?

Question	ECB	CBC	CTR
Chaining	None – blocks independent	Each block depends on previous ciphertext	None – independent counter blocks
Randomisation	None	Random IV	Unique nonce (+counter)
Partial last block	Needs padding	Needs padding	No padding needed

> **Key idea**
> The security of a mode is **not** the security of AES. AES itself is unbroken, yet a bad mode (ECB) or bad usage (reusing a CTR nonce) can leak your data completely.

3. Padding Concepts

3.1 Why padding exists

ECB and CBC feed whole 16-byte blocks into AES. If the message length is not a multiple of 16, the final block is incomplete, so we must **pad** it up to the block boundary. The receiver must be able to remove the padding without ambiguity, so the padding must be self-describing.

3.2 PKCS#7 padding rule

PKCS#7 (also used by PKCS#5 for 8-byte blocks) is the standard scheme. Let *n* be the number of bytes needed to reach the next multiple of the block size (16 for AES). Append *n* bytes, each with the value *n*:

- Number of padding bytes **n = 16 − (len mod 16)**, always between 1 and 16.
- If the message is already a multiple of 16, a **full extra block** of sixteen `0x10` bytes is added. Padding is never zero bytes long; this is what makes removal unambiguous.
- To unpad: read the last byte *n*, check that the last *n* bytes all equal *n*, and drop them.

Message length	len mod 16	Pad bytes n	Pad byte value	Padded length
5	5	11	0x0B	16
15	15	1	0x01	16
16	0	16	0x10	32
20	4	12	0x0C	32
31	15	1	0x01	32

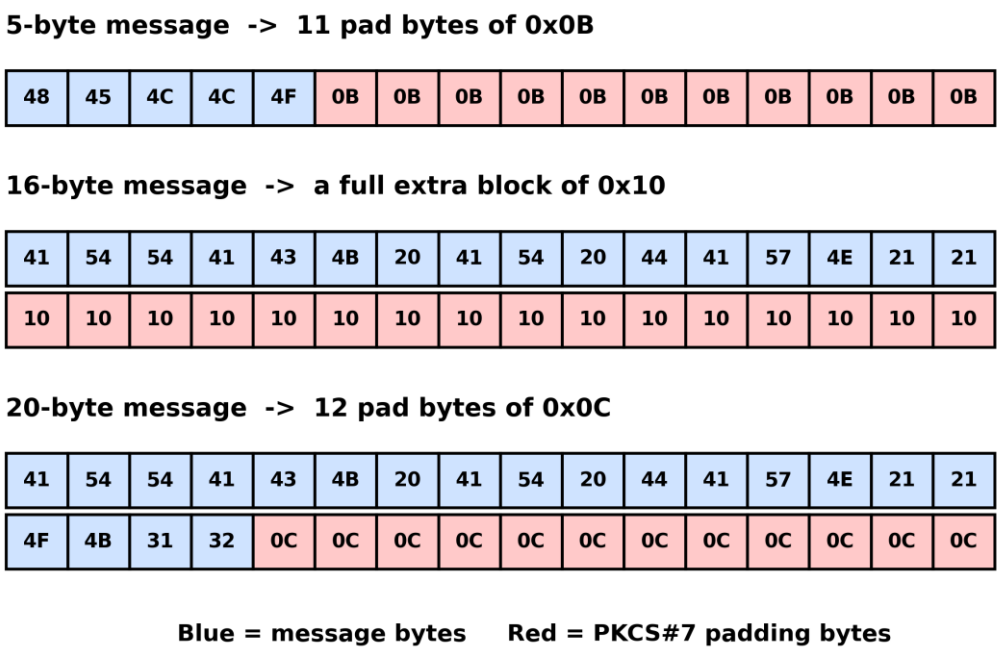


Figure 1: PKCS#7 padding examples (message bytes in blue, padding bytes in red)

3.3 Other padding schemes

Scheme	Padding bytes	Removable?	Remark
PKCS#7	n copies of the value n	Yes, always	De-facto standard; used by the `cryptography` library
ANSI X9.23	zeros, last byte = n	Yes	Only last byte is checked; older banking standard
ISO/IEC 7816-4	0x80 then zeros	Yes	Marker byte makes boundary explicit
Zero padding	0x00 bytes	No if data can end in 0x00	Ambiguous; avoid for binary data

3.4 Runnable code: padding from scratch

The script below implements PKCS#7 padding and unpadding manually, checks that our version equals the library version, shows why zero padding fails, and demonstrates detection of corrupt padding.

Code: padding_demo.py

```
# padding_demo.py - PKCS#7 padding for AES (block size = 16 bytes)
from cryptography.hazmat.primitives import padding

BLOCK = 16  # AES block size in bytes (128 bits)


def pkcs7_pad(data: bytes, block: int = BLOCK) -> bytes:
    pad_len = block - (len(data) % block)      # always 1..16, never 0
    return data + bytes([pad_len]) * pad_len   # every pad byte stores pad_len


def pkcs7_unpad(padded: bytes, block: int = BLOCK) -> bytes:
    if len(padded) == 0 or len(padded) % block != 0:
        raise ValueError("padded data length is not a multiple of the block size")
    pad_len = padded[-1]                       # last byte tells how many to strip
    if pad_len < 1 or pad_len > block:
        raise ValueError("invalid padding length byte")
    if padded[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("padding bytes are corrupt")
    return padded[:-pad_len]


def show_blocks(data: bytes) -> None:
    for i in range(0, len(data), BLOCK):
        print(f"    block {i // BLOCK}: {data[i:i + BLOCK].hex(' ')}")


print("=== 1. PKCS#7 padding for different message lengths ===")
for msg in [b"HELLO", b"ATTACK AT DAWN!!", b"ATTACK AT DAWN!! OK"]:
    padded = pkcs7_pad(msg)
    print(f"message={msg!r} len={len(msg)} -> padded len={len(padded)}, "
          f"pad bytes={len(padded) - len(msg)}")
    show_blocks(padded)

print("\n=== 2. Unpadding restores the original message ===")
padded = pkcs7_pad(b"HELLO")
print("padded  :", padded)
print("unpadded:", pkcs7_unpad(padded))

print("\n=== 3. Manual padding equals the library padder ===")
padder = padding.PKCS7(128).padder()           # 128 = block size in BITS
lib_padded = padder.update(b"HELLO") + padder.finalize()
print("identical to manual version:", lib_padded == pkcs7_pad(b"HELLO"))

print("\n=== 4. Why zero padding is ambiguous ===")
original = b"DATA\x00"                         # genuine trailing zero byte
zero_padded = original + b"\x00" * (BLOCK - len(original) % BLOCK)
```

```
recovered = zero_padded.rstrip(b"\x00")        # receiver strips every zero
print("original :", original)
print("recovered:", recovered)
print("data lost:", recovered != original)

print("\n=== 5. Corrupt padding is detected ===")
bad = padded[:-1] + b"\x07"                    # last byte no longer matches
try:
    pkcs7_unpad(bad)
except ValueError as err:
    print("ValueError:", err)
```

Output (actual, from running the script above)

```
=== 1. PKCS#7 padding for different message lengths ===
message=b'HELLO' len=5 -> padded len=16, pad bytes=11
    block 0: 48 45 4c 4c 4f 0b 0b 0b 0b 0b 0b 0b 0b 0b 0b 0b
message=b'ATTACK AT DAWN!!' len=16 -> padded len=32, pad bytes=16
    block 0: 41 54 54 41 43 4b 20 41 54 20 44 41 57 4e 21 21
    block 1: 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
message=b'ATTACK AT DAWN!! OK' len=19 -> padded len=32, pad bytes=13
    block 0: 41 54 54 41 43 4b 20 41 54 20 44 41 57 4e 21 21
    block 1: 20 4f 4b 0d 0d 0d 0d 0d 0d 0d 0d 0d 0d 0d 0d 0d

=== 2. Unpadding restores the original message ===
padded  : b'HELLO\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b'
unpadded: b'HELLO'

=== 3. Manual padding equals the library padder ===
identical to manual version: True

=== 4. Why zero padding is ambiguous ===
original : b'DATA\x00'
recovered: b'DATA'
data lost: True

=== 5. Corrupt padding is detected ===
ValueError: padding bytes are corrupt
```

Line-by-line explanation

Lines	What they do
1–2	Header comment, then import the library `padding` module. We use it only in Demo 3 to prove our manual code matches the standard implementation.
4	`BLOCK = 16`: AES block size in bytes. (The library, in contrast, wants the size in bits, i.e. 128.)
7–9	`pkcs7_pad`. Line 8 computes `16 - (len % 16)`. The result is 1…16 and never 0, so an already-aligned message gets a whole extra block. Line 9 builds `pad_len` copies of the byte `pad_len` and appends them.
12–20	`pkcs7_unpad`. Lines 13–14 reject inputs that are empty or not block-aligned. Line 15 reads the last byte, which announces the padding length. Lines 16–17 reject impossible values (0 or above 16). Lines 18–19 verify that all pad bytes carry the same value, which detects corruption. Line 20 slices the padding off.
23–25	Helper that prints each 16-byte block as space-separated hex (`hex(' ')`) so padding bytes are easy to see.
28–33	Demo 1. Three messages of 5, 16 and 19 bytes. Note the 16-byte message in the output: block 1 is a complete block of `10` bytes, exactly the “full extra block” rule.
35–38	Demo 2. Pad then unpad `HELLO`; the original is restored.
40–43	Demo 3. `padding.PKCS7(128).padder()` is the library implementation. `update()` consumes data, `finalize()` emits the padded tail. The comparison prints `True`.

Lines	What they do
45–51	Demo 4. The data legitimately ends in a zero byte. Zero padding adds more zeros, and `rstrip` cannot tell which zeros were real, so the original is not recovered (`data lost: True`).
53–58	Demo 5. Overwrite the last byte with `0x07`. The check on line 18 fails and a `ValueError` is raised, showing that PKCS#7 can detect damaged padding.

3.5 Padding pitfalls

- **Length leakage:** padding hides the exact length only up to a 16-byte granularity. Ciphertext length still reveals approximate message length.
- **Padding-oracle attacks:** if a server tells an attacker whether decrypted padding was valid (different error, timing or response), CBC ciphertexts can be decrypted byte by byte without the key. This is why decryption errors must be uniform and why authenticated encryption is recommended (Section 9).
- **Only ECB and CBC need padding.** CTR is a stream-style mode and works on any byte length, so the ciphertext has exactly the plaintext length.

4. ECB: Electronic Codebook Mode

4.1 How ECB works

ECB is the simplest mode: split the (padded) plaintext into 16-byte blocks and encrypt each block **independently** with the same key.

Operation	Formula
Encryption	C_i = E_K(P_i)
Decryption	P_i = D_K(C_i)

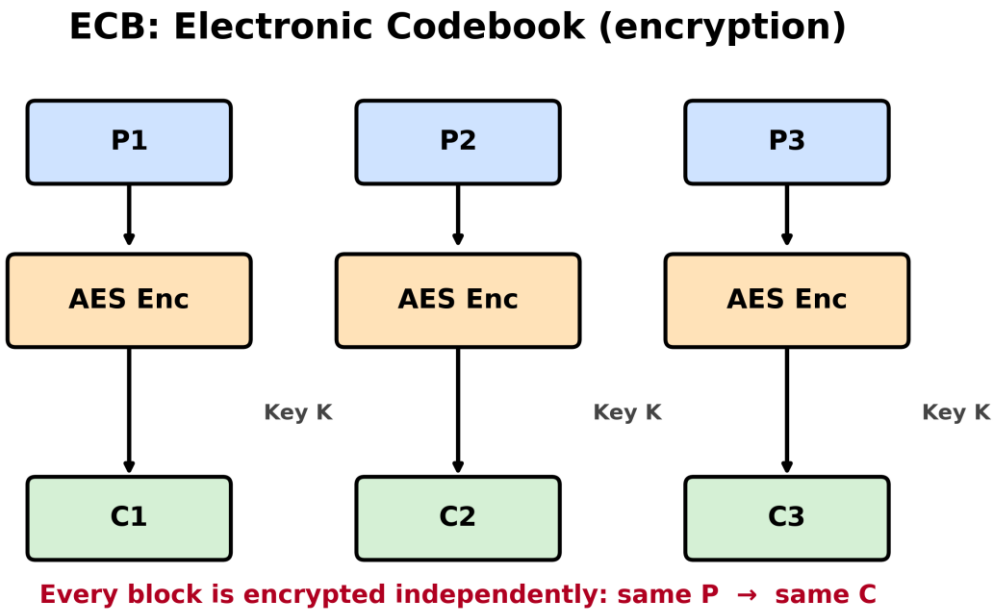


Figure 2: ECB encryption. Every block is processed in isolation.

Properties

- **Deterministic:** the same plaintext block always gives the same ciphertext block (under one key).
- **Parallel and random access:** blocks can be encrypted or decrypted in any order and on many cores.
- **No IV, no chaining, no error propagation beyond one block:** a damaged ciphertext block corrupts only its own plaintext block.
- **Needs padding** for the last block.

4.2 Runnable code: AES-ECB

The script wraps padding and ECB encryption into reusable functions, then runs three experiments: a round trip, repeated-block leakage, and an undetected block reordering.

Code: ecb_demo.py

```
# ecb_demo.py - AES in ECB mode (educational use only - ECB is NOT secure)
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")   # 128-bit demo key


def ecb_encrypt(key: bytes, plaintext: bytes) -> bytes:
    padder = padding.PKCS7(128).padder()
    padded = padder.update(plaintext) + padder.finalize()      # step 1: pad
```

```
11      enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor() # step 2: ECB
12      return enc.update(padded) + enc.finalize()
13
14
15  def ecb_decrypt(key: bytes, ciphertext: bytes) -> bytes:
16      dec = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
17      padded = dec.update(ciphertext) + dec.finalize()           # step 1: decrypt
18      unpadder = padding.PKCS7(128).unpadder()
19      return unpadder.update(padded) + unpadder.finalize()       # step 2: unpad
20
21
22  def blocks(data: bytes):
23      return [data[i:i + 16] for i in range(0, len(data), 16)]
24
25
26  print("=== 1. Encrypt / decrypt round trip ===")
27  message = b"Meet me at the library at 9 PM."
28  ct = ecb_encrypt(KEY, message)
29  print("plaintext :", message, f"({len(message)} bytes)")
30  print("ciphertext:", ct.hex(), f"({len(ct)} bytes)")
31  print("decrypted :", ecb_decrypt(KEY, ct))
32
33  print("\n=== 2. Identical plaintext blocks -> identical ciphertext blocks ===")
34  pt = b"SAMEBLOCK_16BYTE" * 3 + b"DIFFERENT_BLOCK!"
35  ct = ecb_encrypt(KEY, pt)
36  for i, (p, c) in enumerate(zip(blocks(pt), blocks(ct))):
37      print(f"block {i}: P={p.decode()}  C={c.hex()}")
38  print("C0 == C1 == C2 :", blocks(ct)[0] == blocks(ct)[1] == blocks(ct)[2])
39
40  print("\n=== 3. Block reordering goes undetected ===")
41  order = b"PAY TO: ALICE   " + b"AMOUNT: $ 100   " + b"SIGNED: BANK-1  "
42  ct_blocks = blocks(ecb_encrypt(KEY, order))
43  swapped = [ct_blocks[2], ct_blocks[1], ct_blocks[0], ct_blocks[3]]  # attacker
44  print("original :", ecb_decrypt(KEY, b"".join(ct_blocks)))
45  print("tampered :", ecb_decrypt(KEY, b"".join(swapped)))
```

Output (actual, from running the script above)

```
=== 1. Encrypt / decrypt round trip ===
plaintext : b'Meet me at the library at 9 PM.' (31 bytes)
ciphertext: 28a1f499ccc67954d74b8a2b7d9045969915768f7a887addc7bbd6a03a56c88b (32 bytes)
decrypted : b'Meet me at the library at 9 PM.'

=== 2. Identical plaintext blocks -> identical ciphertext blocks ===
block 0: P=SAMEBLOCK_16BYTE  C=7349c6849ee776941e4c68f297afb990
block 1: P=SAMEBLOCK_16BYTE  C=7349c6849ee776941e4c68f297afb990
block 2: P=SAMEBLOCK_16BYTE  C=7349c6849ee776941e4c68f297afb990
block 3: P=DIFFERENT_BLOCK!  C=ff849134668f496bd7db4faec254631d
C0 == C1 == C2 : True

=== 3. Block reordering goes undetected ===
original : b'PAY TO: ALICE   AMOUNT: $ 100   SIGNED: BANK-1  '
tampered : b'SIGNED: BANK-1  AMOUNT: $ 100   PAY TO: ALICE   '
```

Line-by-line explanation

Lines	What they do
1–3	Header and imports. `Cipher`, `algorithms`, `modes` build the cipher; `padding` is used for PKCS#7.
5	A fixed 16-byte (AES-128) demo key built from hex. Real keys must come from a secure random source.
8–12	`ecb_encrypt`: create a PKCS#7 padder (line 9), pad (line 10), build an AES-ECB encryptor (line 11), encrypt and flush (line 12). `modes.ECB()` takes no IV or nonce.
15–19	`ecb_decrypt` performs the steps in reverse order: decrypt first (line 17), then remove padding (lines 18–19). Unpadding is always done after decryption.
22–23	Helper that slices data into 16-byte blocks so we can compare them.

Lines	What they do
26–31	Demo 1. The 31-byte message is padded by 1 byte to 32 bytes (two blocks), encrypted, and decrypted back correctly.
33–38	Demo 2. Three identical 16-byte blocks followed by a different one. In the output blocks 0, 1 and 2 have the same ciphertext `7349c684…`, so `C0 == C1 == C2` is `True`. An eavesdropper learns which blocks repeat without knowing the key.
40–45	Demo 3. A three-block “payment order”. Line 43 swaps ciphertext blocks 0 and 2 as an attacker would. Decryption still succeeds and returns a valid-looking but rearranged message. ECB offers no integrity and no ordering protection.

4.3 Seeing the leak: encrypting an image

The classic way to see ECB's flaw is to encrypt raw pixel data. Because identical pixel blocks produce identical ciphertext blocks, the outline of the picture survives. The script below encrypts the same 128×128 image with ECB, CBC and CTR.

Code: ecb_image_demo.py

```
# ecb_image_demo.py - why ECB leaks structure (visual proof)
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
IV = bytes.fromhex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf")
CTR0 = bytes.fromhex("f0f1f2f3f4f5f6f7") + bytes(8)

# 128 x 128 grayscale picture: white background, black ring and diagonal bar
y, x = np.mgrid[0:128, 0:128]
ring = (np.hypot(x - 64, y - 64) < 48) & (np.hypot(x - 64, y - 64) > 30)
bar = abs(x - y) < 6
img = np.full((128, 128), 255, dtype=np.uint8)
img[ring | bar] = 0
raw = img.tobytes()                        # 16384 bytes = 1024 AES blocks (no padding)


def encrypt(mode):
    op = Cipher(algorithms.AES(KEY), mode).encryptor()
    return np.frombuffer(op.update(raw) + op.finalize(), dtype=np.uint8)


panels = [("Original", img),
          ("ECB", encrypt(modes.ECB()).reshape(128, 128)),
          ("CBC", encrypt(modes.CBC(IV)).reshape(128, 128)),
          ("CTR", encrypt(modes.CTR(CTR0)).reshape(128, 128))]

fig, axes = plt.subplots(1, 4, figsize=(12, 3.6), facecolor="white")
for ax, (title, data) in zip(axes, panels):
    ax.imshow(data, cmap="gray", vmin=0, vmax=255)
    ax.set_title(title, fontsize=16, fontweight="bold")
    ax.axis("off")
fig.tight_layout()
fig.savefig("fig_ecb_image.png", dpi=300, facecolor="white")
print("saved fig_ecb_image.png")
```

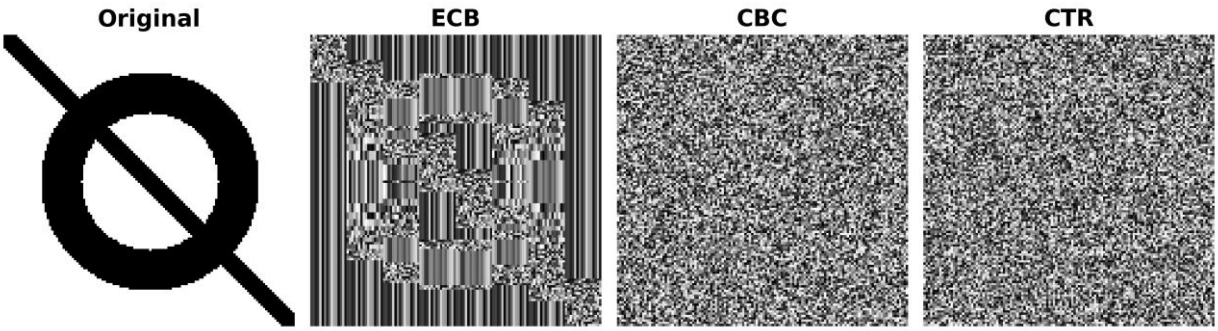


Figure 3: The same image encrypted with ECB (structure visible), CBC and CTR (random noise)

Line-by-line explanation

Lines	What they do
2–6	Import NumPy and Matplotlib (with the non-interactive `Agg` backend so it runs without a display) and the cipher classes.
8–10	Demo key, a CBC IV, and a 16-byte initial counter block for CTR (8-byte nonce + 8 zero bytes).
12–18	Draw a white 128×128 grayscale image with a black ring and a diagonal bar. `tobytes()` yields 16,384 bytes, which is exactly 1,024 AES blocks, so no padding is needed.
21–23	`encrypt(mode)` encrypts the raw pixel bytes with the chosen mode and converts the result back to a NumPy array.
26–29	Four panels: original, ECB, CBC, CTR, each reshaped back to 128×128.
31–37	Draw them side by side on a pure white background and save at 300 DPI.

4.4 Verdict on ECB

> **Security warning**
>
> Do **not** use ECB to encrypt anything longer than one block, or any data with structure. It leaks equality of blocks, allows cut-and-paste of blocks, and gives no integrity. Its legitimate uses are tiny: building blocks inside other constructions (key wrapping, for example).

5. CBC: Cipher Block Chaining Mode

5.1 How CBC works

CBC removes ECB's determinism by **chaining**: before encrypting, each plaintext block is XORed with the previous ciphertext block. The very first block is XORed with a random **Initialization Vector (IV)**.

Operation	Formula
Encryption	$C_0 = IV;\quad C_i = E_K(P_i \oplus C_{i-1})$
Decryption	$P_i = D_K(C_i) \oplus C_{i-1}$

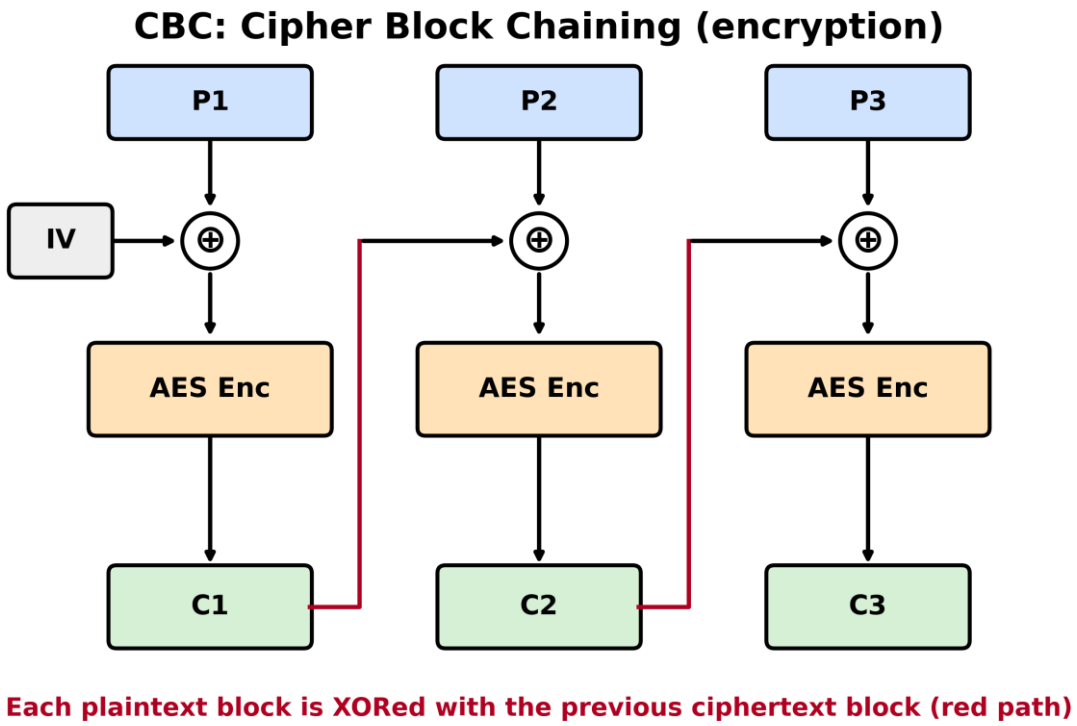


Figure 4: CBC encryption. The red path carries each ciphertext block into the next XOR.

The IV

- Must be **16 bytes**, and **unpredictable (random)** for every message encrypted under the same key.
- Need **not** be secret. It is normally sent in front of the ciphertext (see Section 8).
- Reusing an IV re-introduces leakage: two messages starting with the same blocks would produce identical leading ciphertext blocks.

Properties

- **Encryption is sequential** (block *i* needs block *i−1*), so it cannot be parallelised.
- **Decryption is parallelisable**, because every C_i is already known.
- **Error propagation:** a flipped bit in C_i garbles all of P_i and flips the *same bit* in P_(i+1). Later blocks are unaffected (the mode is self-synchronising).
- **Needs padding.** Provides confidentiality only, **not** integrity.

5.2 Runnable code: AES-CBC

Four experiments: IV influence, rebuilding CBC manually from raw AES, a bit-flipping attack, and the effect of a wrong IV.

Code: cbc_demo.py

```
# cbc_demo.py - AES in CBC mode
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
IV1 = bytes.fromhex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf")    # demo IV only
IV2 = bytes.fromhex("b0b1b2b3b4b5b6b7b8b9babbbcbdbebf")    # a different IV


def cbc_encrypt(key, iv, plaintext):
    padder = padding.PKCS7(128).padder()
    padded = padder.update(plaintext) + padder.finalize()
    enc = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
    return enc.update(padded) + enc.finalize()


def cbc_decrypt(key, iv, ciphertext):
    dec = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
    padded = dec.update(ciphertext) + dec.finalize()
    unpadder = padding.PKCS7(128).unpadder()
    return unpadder.update(padded) + unpadder.finalize()


def blocks(data):
    return [data[i:i + 16] for i in range(0, len(data), 16)]


def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))


print("=== 1. Round trip and the role of the IV ===")
pt = b"SAMEBLOCK_16BYTE" * 3 + b"DIFFERENT_BLOCK!"
ct1 = cbc_encrypt(KEY, IV1, pt)
ct2 = cbc_encrypt(KEY, IV2, pt)
for i, c in enumerate(blocks(ct1)):
    print(f"IV1 block {i}: {c.hex()}")
print("same plaintext, different IV -> different ciphertext:", ct1 != ct2)
print("repeated blocks now equal? :", blocks(ct1)[0] == blocks(ct1)[1])
print("decrypted OK:", cbc_decrypt(KEY, IV1, ct1) == pt)

print("\n=== 2. Rebuilding CBC by hand: C[i] = E(P[i] xor C[i-1]) ===")
raw = Cipher(algorithms.AES(KEY), modes.ECB()).encryptor()   # bare block cipher
padder = padding.PKCS7(128).padder()
padded = padder.update(pt) + padder.finalize()
prev, manual = IV1, b""
for p in blocks(padded):
    c = raw.update(xor(p, prev))      # XOR with previous ciphertext, then AES
    manual += c
    prev = c                          # chain: this ciphertext feeds the next block
print("manual chain equals library:", manual == ct1)

print("\n=== 3. Bit-flipping: damage caused by editing ciphertext ===")
msg = b"PAY TO: ALICE   " + b"AMOUNT: $ 100   "
ct = cbc_encrypt(KEY, IV1, msg)
tampered = bytearray(ct)
tampered[10] ^= ord("1") ^ ord("9")        # edit byte 10 of C0
res = cbc_decrypt(KEY, IV1, bytes(tampered))
print("P0 (garbled):", res[:16])
print("P1 (chosen) :", res[16:32])

print("\n=== 4. Wrong IV only damages the first block ===")
res = cbc_decrypt(KEY, IV2, ct1)
print("P0 correct?", res[:16] == pt[:16], "| P1..P3 correct?", res[16:] == pt[16:])
```

Output (actual, from running the script above)

```
=== 1. Round trip and the role of the IV ===
IV1 block 0: 121dfebee29d40c31925a4d96dccace2
IV1 block 1: 8b73903c9a1d116679d459138d9b4286
IV1 block 2: dd824cf2380015323f92e73ddc2990ab
IV1 block 3: d3791900969635b782f245e5a8bb4b09
IV1 block 4: 15f77f8521af1532e486286734b27993
same plaintext, different IV -> different ciphertext: True
repeated blocks now equal? : False
decrypted OK: True

=== 2. Rebuilding CBC by hand: C[i] = E(P[i] xor C[i-1]) ===
manual chain equals library: True

=== 3. Bit-flipping: damage caused by editing ciphertext ===
P0 (garbled): b'\xd3\x9d\xfd\xee\x14=\x18\x83q\x10\xd5\xd2B\x8a\x9aL'
P1 (chosen) : b'AMOUNT: $ 900   '

=== 4. Wrong IV only damages the first block ===
P0 correct? False | P1..P3 correct? True
```

Line-by-line explanation

Lines	What they do
1–3	Header and imports (same toolkit as ECB).
5–7	One demo key and two different 16-byte demo IVs, `IV1` and `IV2`, so we can compare encryptions.
10–14	`cbc_encrypt(key, iv, plaintext)`: PKCS#7 pad (lines 11–12), then `modes.CBC(iv)` (line 13) and encrypt (line 14). The IV is a parameter, never a hidden global.
17–21	`cbc_decrypt`: decrypt with the same IV, then unpad. The decryptor needs the exact IV used at encryption.
24–29	`blocks()` splits data into 16-byte blocks; `xor()` XORs two equal-length byte strings byte by byte.
32–40	Demo 1. The same 64-byte plaintext as in the ECB demo (three identical blocks). Output shows five ciphertext blocks (four data blocks + one full padding block) that are all different; line 39 prints `False` for “repeated blocks equal”. A different IV changes the whole ciphertext, and decryption with the right IV is correct.
42–51	Demo 2: implement the formula by hand. `raw` is a bare AES-ECB encryptor, i.e. just E_K. Line 48 does XOR with the previous ciphertext then AES; line 50 makes this ciphertext the “previous” value for the next block (`prev` starts as the IV). The result equals the library output.
53–60	Demo 3: bit-flipping. A two-block message becomes three ciphertext blocks (the third is the padding block). Line 57 XORs byte 10 of C0 with `'1' ⊕ '9'`. In decryption P1 = D_K(C1) ⊕ C0, so byte 10 of P1 turns from `1` to `9` exactly as chosen (`$ 900`), while P0, decrypted from the damaged C0, becomes random garbage. An attacker can edit one block of plaintext at the price of destroying the one before it.
62–64	Demo 4. Decrypting with `IV2` instead of `IV1` corrupts only block 0 (`P0 correct? False`); everything after is right because later blocks depend on ciphertext blocks, not on the IV.

5.3 Verdict on CBC

CBC was the workhorse mode of TLS 1.0–1.2 and many file formats. It is secure for confidentiality **if** the IV is random and padding errors are never exposed. Because it is malleable (Demo 3) and vulnerable to padding oracles, modern designs prefer authenticated modes.

6. CTR: Counter Mode

6.1 How CTR works

CTR turns AES into a **stream cipher**. Instead of encrypting the message, AES encrypts a sequence of counter blocks (nonce ‖ counter) to produce a pseudo-random **keystream**, which is XORed with the plaintext.

Operation	Formula
Keystream	S_i = E_K(Nonce ‖ Counter_i), Counter_i = i
Encryption	C_i = P_i ⊕ S_i
Decryption	P_i = C_i ⊕ S_i (identical operation)

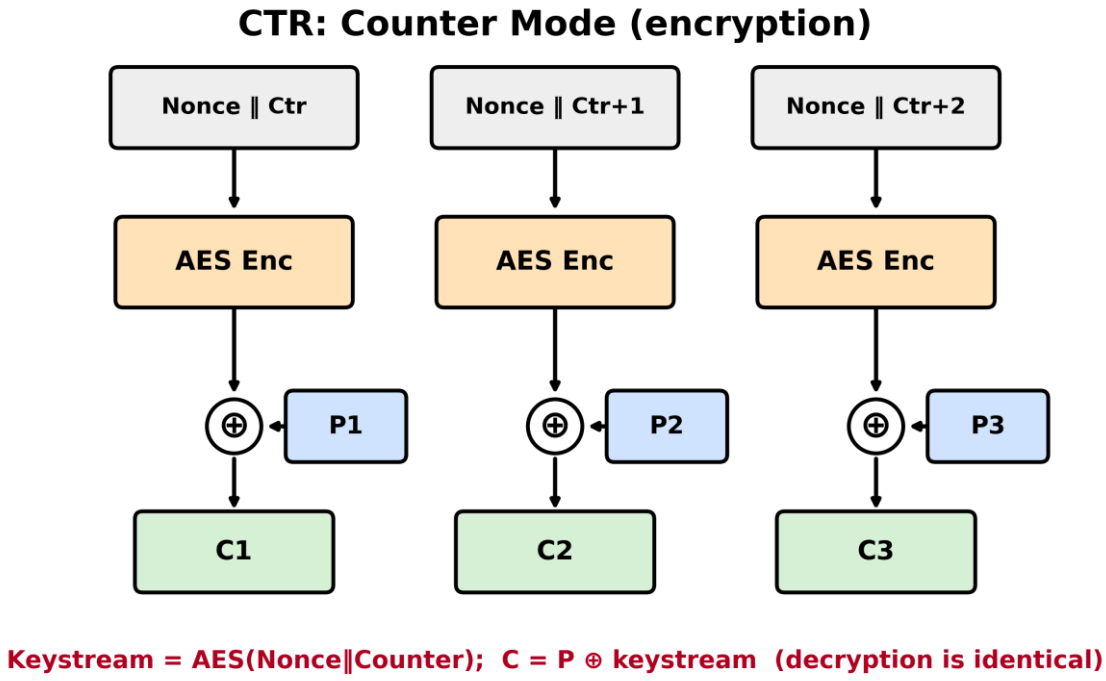


Figure 5: CTR encryption. AES only ever encrypts counter blocks.

Properties

- **No padding:** ciphertext length equals plaintext length (the last keystream block is simply truncated).
- **Fully parallel** encryption and decryption, and **random access**: block *i* can be decrypted by itself by jumping to counter *i*.
- **AES decryption is never used**; both directions only need E_K.
- **Identical plaintext blocks give different ciphertext**, because every counter block is different.
- **Nonce uniqueness is critical.** The pair (key, nonce+counter) must never repeat.
- **Malleable:** a flipped ciphertext bit flips exactly the same plaintext bit. Integrity must be added separately.

6.2 Runnable code: AES-CTR

Six experiments: lengths without padding, manual keystream, pattern hiding, random access, a nonce-reuse attack and malleability. In this program the 16-byte counter block is an 8-byte nonce followed by an 8-byte big-endian counter.

Code: ctr_demo.py

```
# ctr_demo.py - AES in CTR mode (a stream cipher built from a block cipher)
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
NONCE = bytes.fromhex("f0f1f2f3f4f5f6f7")      # 8-byte nonce (demo value)


def counter_block(nonce: bytes, n: int) -> bytes:
    return nonce + n.to_bytes(8, "big")         # 16 bytes = nonce || counter


def ctr_crypt(key, nonce, data, start=0):
    # Encryption and decryption are the SAME operation in CTR mode
    cipher = Cipher(algorithms.AES(key), modes.CTR(counter_block(nonce, start)))
    op = cipher.encryptor()
    return op.update(data) + op.finalize()


def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))


print("=== 1. No padding: ciphertext length equals plaintext length ===")
msg = b"Meet me at the library at 9 PM."          # 31 bytes
ct = ctr_crypt(KEY, NONCE, msg)
print("plaintext length :", len(msg))
print("ciphertext length:", len(ct))
print("ciphertext       :", ct.hex())
print("decrypted        :", ctr_crypt(KEY, NONCE, ct))

print("\n=== 2. Rebuilding CTR by hand: C = P xor E(nonce||counter) ===")
raw = Cipher(algorithms.AES(KEY), modes.ECB()).encryptor()
keystream = b"".join(raw.update(counter_block(NONCE, i)) for i in range(2))
print("keystream block 0:", keystream[:16].hex())
print("keystream block 1:", keystream[16:].hex())
print("manual result equals library:", xor(msg, keystream) == ct)

print("\n=== 3. Identical blocks give different ciphertext blocks ===")
pt = b"SAMEBLOCK_16BYTE" * 3
ct3 = ctr_crypt(KEY, NONCE, pt)
print([ct3[i:i + 16].hex()[:12] + "..." for i in range(0, 48, 16)])

print("\n=== 4. Random access: decrypt only block 2 ===")
full = ctr_crypt(KEY, NONCE, pt)
only_b2 = ctr_crypt(KEY, NONCE, full[32:48], start=2)   # jump straight to counter 2
print("block 2 recovered alone:", only_b2)

print("\n=== 5. DANGER: nonce reuse leaks plaintext ===")
p1 = b"ATTACK AT DAWN!!"
p2 = b"RETREAT AT NOON!"
c1 = ctr_crypt(KEY, NONCE, p1)
c2 = ctr_crypt(KEY, NONCE, p2)                          # same key + same nonce
print("C1 xor C2 == P1 xor P2 :", xor(c1, c2) == xor(p1, p2))
print("attacker knows P1 -> P2:", xor(xor(c1, c2), p1))

print("\n=== 6. Malleability: bit-flips pass straight through ===")
c = bytearray(ctr_crypt(KEY, NONCE, b"AMOUNT: $ 100   "))
c[10] ^= ord("1") ^ ord("9")
print("tampered plaintext:", ctr_crypt(KEY, NONCE, bytes(c)))
```

Output (actual, from running the script above)

```
=== 1. No padding: ciphertext length equals plaintext length ===
plaintext length : 31
ciphertext length: 31
```

```
ciphertext       : 5a00d7cb8468907a6dc36d9bb94961d31f721d76848162224d6e4fa1c9f77d
decrypted        : b'Meet me at the library at 9 PM.'

=== 2. Rebuilding CTR by hand: C = P xor E(nonce||counter) ===
keystream block 0: 1765b2bfa405f55a0cb74defd12c41bf
keystream block 1: 76106f17f6f84243394e768199ba538b
manual result equals library: True

=== 3. Identical blocks give different ciphertext blocks ===
['4424fffae649...', '25512252b4b4...', 'f9cbe2edb233...']

=== 4. Random access: decrypt only block 2 ===
block 2 recovered alone: b'SAMEBLOCK_16BYTE'

=== 5. DANGER: nonce reuse leaks plaintext ===
C1 xor C2 == P1 xor P2 : True
attacker knows P1 -> P2: b'RETREAT AT NOON!'

=== 6. Malleability: bit-flips pass straight through ===
tampered plaintext: b'AMOUNT: $ 900   '
```

Line-by-line explanation

Lines	What they do
4–5	Demo key and an 8-byte nonce (the first half of the counter block).
8–9	`counter_block` builds the 16-byte value `nonce ‖ counter`; `to_bytes(8, 'big')` writes the counter as 8 big-endian bytes.
12–16	`ctr_crypt` does both encryption and decryption. `modes.CTR(...)` receives the initial counter block; the library increments it for every 16 bytes. The `start` argument lets us begin at any counter value.
19–20	Byte-wise XOR helper.
23–29	Demo 1. A 31-byte message gives a 31-byte ciphertext, with no padding and no expansion. Applying `ctr_crypt` again recovers the plaintext.
31–36	Demo 2. A bare AES-ECB object encrypts counter blocks 0 and 1 to build the keystream (lines 32–33). XORing it with the message reproduces the library ciphertext (`True`), proving CTR is just “plaintext ⊕ AES(counter)”.
38–41	Demo 3. Three identical blocks encrypt to three different values, so the ECB pattern leak is gone.
43–46	Demo 4. Take only ciphertext bytes 32–48 and call `ctr_crypt` with `start=2`. Block 2 is recovered without touching blocks 0 and 1, which makes CTR attractive for disk and database encryption.
48–54	Demo 5: nonce reuse. Two messages are encrypted with the same key and nonce, so the keystream is identical and cancels out: `C1 ⊕ C2 = P1 ⊕ P2`. If the attacker knows or guesses P1 (line 54), P2 is recovered completely, with no key needed.
56–59	Demo 6: malleability. Flipping a ciphertext bit changes exactly the corresponding plaintext bit (`$ 100` → `$ 900`) with no collateral garbling, which is even easier than CBC bit-flipping.

6.3 Verdict on CTR

CTR is fast, parallel and padding-free, and is the confidentiality engine inside AES-GCM. By itself it provides no integrity, and a repeated nonce is catastrophic. Use it with an authentication tag or via a ready-made authenticated mode.

7. Side-by-Side Comparison

7.1 Property table

Property	ECB	CBC	CTR
Padding required	Yes	Yes	No
IV / nonce	None	16-byte random IV	Unique nonce (e.g. 8–12 bytes) + counter
Same plaintext block →	Same ciphertext block	Different (IV chaining)	Different (counter)
Pattern leakage	Severe	None for random IV	None if nonce unique
Encrypt in parallel	Yes	No	Yes
Decrypt in parallel	Yes	Yes	Yes
Random access to blocks	Yes	Decrypt: yes; encrypt: no	Yes
AES decrypt function needed	Yes	Yes	No
Ciphertext expansion	1–16 bytes (padding)	1–16 bytes + 16-byte IV	0 bytes + nonce
Bit-flip effect	Garbles that block only	Garbles that block, flips bit in next	Flips exactly that bit
Integrity	None	None	None
Main risk	Structure leakage	Padding oracle, IV reuse	Nonce reuse
Recommended?	No	Legacy only	Yes, with authentication (GCM)

7.2 Runnable code: all three on the same input

The final comparison encrypts one message containing four identical 16-byte blocks plus a 4-byte tail with each mode, then reports ciphertext length, padding overhead and how many *distinct* ciphertext blocks appear.

Code: compare_modes.py

```
# compare_modes.py - ECB vs CBC vs CTR on the same input
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
IV = bytes.fromhex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf")
NONCE_BLOCK = bytes.fromhex("f0f1f2f3f4f5f6f7") + bytes(8)


def pad(data):
    p = padding.PKCS7(128).padder()
    return p.update(data) + p.finalize()


def run(mode, data):
    op = Cipher(algorithms.AES(KEY), mode).encryptor()
    return op.update(data) + op.finalize()


def unique_blocks(ct):
    return len({ct[i:i + 16] for i in range(0, len(ct), 16)})


msg = b"ABCDEFGHIJKLMNOP" * 4 + b"tail"       # 4 identical blocks + 4 bytes
results = {
```

```
26      "ECB": run(modes.ECB(), pad(msg)),
27      "CBC": run(modes.CBC(IV), pad(msg)),
28      "CTR": run(modes.CTR(NONCE_BLOCK), msg),
29  }
30  print(f"plaintext length = {len(msg)} bytes, identical 16-byte blocks = 4")
31  print(f"{'Mode':<5}{'CT length':>10}{'Padding bytes':>15}{'Unique CT blocks':>18}")
32  for name, ct in results.items():
33      extra = len(ct) - len(msg) if name != "CTR" else 0
34      print(f"{name:<5}{len(ct):>10}{extra:>15}{unique_blocks(ct):>18}")
```

Output (actual, from running the script above)

```
plaintext length = 68 bytes, identical 16-byte blocks = 4
Mode  CT length  Padding bytes  Unique CT blocks
ECB          80            12                 2
CBC          80            12                 5
CTR          68             0                 5
```

Line-by-line explanation

Lines	What they do
5–7	Demo key, a CBC IV, and the CTR initial counter block (8-byte nonce + 8 zero bytes).
10–12	`pad()`: PKCS#7 padding helper.
15–17	`run(mode, data)`: generic encryptor, so the same function can run any mode object.
20–21	`unique_blocks()` puts every 16-byte ciphertext block into a Python `set` and counts distinct ones; a lower number means more leakage.
24	The message is 68 bytes: four identical blocks `ABCDEFGHIJKLMNOP` plus the tail `tail`.
25–29	ECB and CBC are given padded data; CTR gets the raw message.
30–34	Print a table. Reading the output: ECB and CBC both expand to 80 bytes (12 padding bytes) while CTR stays at 68. ECB shows only 2 unique blocks out of 5 (the four repeats collapse into one), but CBC and CTR show 5 unique blocks, so no repetition is visible.

8. Production Pattern: Fresh IV/Nonce per Message

In real software the key comes from a secure key-management process and the IV or nonce is generated randomly for every message with `os.urandom`. Because the IV/nonce is not secret, it is **prepended to the ciphertext** and split off by the receiver.

Code: random_iv_demo.py

```
1  # random_iv_demo.py - the production pattern: fresh random IV / nonce every time
2  import os
3  from cryptography.hazmat.primitives import padding
4  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
5
6  key = os.urandom(32)                        # random 256-bit AES key
7  msg = b"Meet me at the library at 9 PM."
8
9
10 def cbc_pack(key, plaintext):
11     iv = os.urandom(16)                     # new random IV for EVERY message
12     padder = padding.PKCS7(128).padder()
13     data = padder.update(plaintext) + padder.finalize()
14     enc = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
15     return iv + enc.update(data) + enc.finalize()       # ship IV with ciphertext
16
17
18 def cbc_unpack(key, packet):
19     iv, body = packet[:16], packet[16:]     # split IV from ciphertext
20     dec = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
21     unpadder = padding.PKCS7(128).unpadder()
22     return unpadder.update(dec.update(body) + dec.finalize()) + unpadder.finalize()
23
24
25 def ctr_pack(key, plaintext):
26     nonce = os.urandom(8)                   # new random 64-bit nonce per message
27     enc = Cipher(algorithms.AES(key), modes.CTR(nonce + bytes(8))).encryptor()
28     return nonce + enc.update(plaintext) + enc.finalize()
29
30
31 def ctr_unpack(key, packet):
32     nonce, body = packet[:8], packet[8:]
33     dec = Cipher(algorithms.AES(key), modes.CTR(nonce + bytes(8))).decryptor()
34     return dec.update(body) + dec.finalize()
35
36
37 p1, p2 = cbc_pack(key, msg), cbc_pack(key, msg)
38 print("CBC packet length   :", len(p1), "(16 IV + 32 ciphertext)")
39 print("two CBC encryptions differ:", p1 != p2)
40 print("CBC round trip OK   :", cbc_unpack(key, p1) == msg)
41
42 q1, q2 = ctr_pack(key, msg), ctr_pack(key, msg)
43 print("CTR packet length   :", len(q1), "(8 nonce + 31 ciphertext)")
44 print("two CTR encryptions differ:", q1 != q2)
45 print("CTR round trip OK   :", ctr_unpack(key, q1) == msg)
```

Output (actual, from running the script above)

```
CBC packet length   : 48 (16 IV + 32 ciphertext)
two CBC encryptions differ: True
CBC round trip OK   : True
CTR packet length   : 39 (8 nonce + 31 ciphertext)
two CTR encryptions differ: True
CTR round trip OK   : True
```

Line-by-line explanation

Lines	What they do
2, 6	`os.urandom` is the operating system's cryptographically secure random generator. Line 6 creates a random 256-bit AES key.

Lines	What they do
10–15	`cbc_pack`: new random IV for every message (line 11), pad, encrypt, and return `iv + ciphertext` so the receiver can find the IV.
18–22	`cbc_unpack`: the first 16 bytes are the IV, the rest is ciphertext. Decrypt and unpad.
25–28	`ctr_pack`: a random 8-byte nonce, with the other 8 counter bytes starting at zero (line 27). No padding. The nonce is prepended.
31–34	`ctr_unpack`: split nonce and body, then decrypt with the same keystream.
37–45	Encrypt the same message twice with each mode. The two packets differ (`True`) because the IV/nonce differs, and every round trip succeeds. CBC packets are 48 bytes (16 IV + 32 padded ciphertext); CTR packets are 39 bytes (8 nonce + 31).

> **Tip**
> With a random 8-byte nonce, collisions become likely after roughly 2^32 messages under one key (birthday bound). High-volume systems should use a counter-based nonce or a mode with a larger nonce, such as AES-GCM with a 96-bit nonce and strict key rotation.

9. Choosing a Mode in Practice

9.1 Confidentiality is not enough

Demos 3 (CBC) and 6 (CTR) showed that an attacker can modify ciphertext in predictable ways, and ECB allowed reordering. None of the three modes detects tampering. Real systems need **authenticated encryption**, which also produces a tag that fails verification if anything changes.

Need	Recommended choice
General-purpose encryption of messages / files	AES-GCM (CTR encryption + built-in authentication tag) or ChaCha20-Poly1305
Must use CBC (legacy protocol)	CBC with random IV plus a MAC over IV‖ciphertext, verified before decrypting (Encrypt-then-MAC)
Random-access disk encryption	AES-XTS (a tweakable mode designed for storage)
Anything with structured data	Never ECB

9.2 Common mistakes

Mistake	Consequence	Fix
Using ECB for files or messages	Patterns and repeated blocks visible; blocks can be rearranged	Use CBC/CTR at minimum; preferably AES-GCM
Fixed or reused IV in CBC	Leaks equality of message prefixes	New random 16-byte IV for each message
Reusing a CTR nonce	Keystream reuse: XOR of plaintexts leaks, known plaintext reveals the rest	Unique nonce per (key, message); rotate keys
Zero padding	Trailing zero bytes lost or ambiguous	Use PKCS#7
Revealing padding errors	Padding-oracle attack decrypts CBC data	Authenticate first; return uniform errors
Hard-coded keys	Anyone with the code has the key	Use key management, environment secrets, KDFs
Encryption without integrity	Silent tampering (bit flipping)	Authenticated encryption or Encrypt-then-MAC

10. Practice Exercises

10.1 Questions

1. What padding bytes are added to a 31-byte message and to a 32-byte message under PKCS#7? Give both the count and the hex value.
2. Implement ISO/IEC 7816-4 padding (append `0x80`, then zero bytes up to the block boundary) with a matching unpad function. Verify the round trip for message lengths 0 to 40.
3. In `ecb_demo.py`, change the plaintext in Demo 2 so that only blocks 1 and 3 are identical. Predict which ciphertext blocks will match, then run it.
4. In `cbc_demo.py` Demo 3, change the attack so that `$ 100` becomes `$ 500`. Which byte index and which XOR value do you use?
5. In `cbc_demo.py`, flip one bit in the **last** ciphertext block (the padding block) before decrypting. What happens and why?
6. Modify `ctr_demo.py` so that `NONCE` is changed between the two messages in Demo 5. Does the XOR relationship still hold? Explain.
7. Encrypt a 1000-byte message with AES-256 in CBC and in CTR. Predict both ciphertext lengths, then verify with `len()`.
8. Write a function `ecb_detect(ciphertext)` that returns `True` if any 16-byte block repeats. Test it on ECB, CBC and CTR output.

10.2 Answer key (short)

Q	Answer
1	31 bytes → 1 byte `0x01`; 32 bytes → 16 bytes of `0x10` (a full extra block).
2	Pad: add `0x80` and then zeros until the length is a multiple of 16 (a full extra block when already aligned). Unpad: strip trailing `0x00` bytes, then require the last remaining byte to be `0x80` and remove it.
3	Ciphertext blocks 1 and 3 are equal; blocks 0 and 2 are different from them and from each other.
4	Index 10 of C0; XOR value `ord('1') ^ ord('5')` = 4.
5	Decryption of the final block is corrupted, so unpadding almost certainly fails with a padding error. This is the behaviour padding-oracle attacks exploit.
6	No. With different nonces the keystreams differ, so C1 ⊕ C2 no longer equals P1 ⊕ P2.
7	CBC: 1008 bytes (1000 + 8 padding bytes; add 16 more if the IV is prepended). CTR: 1000 bytes (+ nonce if prepended).
8	Split into 16-byte blocks, compare `len(set(blocks)) < len(blocks)`. Returns `True` only for ECB when the plaintext had repeats.

11. Summary and References

11.1 One-page summary

	ECB	CBC	CTR
Idea	Encrypt blocks independently	XOR with previous ciphertext, then encrypt	Encrypt counters, XOR with plaintext
Formula	C_i = E_K(P_i)	C_i = E_K(P_i ⊕ C_(i−1))	C_i = P_i ⊕ E_K(Nonce‖i)
Padding	Yes (PKCS#7)	Yes (PKCS#7)	No
Randomiser	None	Random IV	Unique nonce
Biggest flaw	Pattern leakage	Padding oracle, malleable	Nonce reuse, malleable
Use today?	No	Legacy	Via AES-GCM

> **Key idea**
> Remember the three rules: (1) never use ECB for real data; (2) never reuse an IV or nonce under the same key; (3) encryption without authentication is incomplete, so prefer AES-GCM.

11.2 References

- NIST FIPS 197, *Advanced Encryption Standard (AES)*, 2001.
- NIST SP 800-38A, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, 2001 (defines ECB, CBC and CTR).
- NIST SP 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, 2007.
- Nielson, S. J., Monson, C. K., *Practical Cryptography in Python*, Apress, 2019.
- Python `cryptography` library documentation, symmetric encryption and padding sections (cryptography.io).